

DBMS_Lab_Final_Notes

1. Create Table is Basic Stuff

2. Drop Table:

```
DROP TABLE table_name;  
DROP TABLE table_name CASCADE CONSTRAINTS;
```

3. Altering Tables:

1. To add a new attribute to the table, we use:

```
1. ALTER TABLE table_name ADD attribute_name datatype;
```

2. We can even add multiple attributes at the same time:

```
1. ALTER TABLE table_name ADD (attribute1 datatype, ...);
```

3. To delete an attribute from a table, we use:

```
1. ALTER TABLE table_name DROP COLUMN attribute_name;
```

4. We can even delete multiple attributes at the same time:

```
1. ALTER TABLE table_name DROP COLUMN (attribute1, ...);
```

5. To modify the data type of an attribute, we need to ensure that the column is empty:

```
1. ALTER TABLE table_name MODIFY attribute_name new_datatype;
```

6. To rename an attribute, we use:

```
1. ALTER TABLE table_name RENAME COLUMN old_attribute_name TO new_attribute_name;
```

7. To rename a table, we use:

```
1. ALTER TABLE table_name RENAME TO new_table_name;
```

4. Inserting records:

```
INSERT INTO TABLE_NAME VALUES (... , ... , ...);  
INSERT INTO CITIZEN VALUES (2015001, 'W', 19, 'Bangladesh');  
INSERT INTO CITIZEN(NATIONAL_ID, NAME, AGE, COUNTRY) VALUES (2015002, 'X', 23, 'Bangladesh');
```

5. Updating Existing Data:

```
UPDATE CITIZEN  
SET NAME = 'Hisenberg', COUNTRY= 'USA'  
WHERE NATIONAL_ID = 1; (WHERE <CONDITIONAL CLAUSE>;)
```

6. Deleting data from table:

```
DELETE FROM TABLE_NAME  
WHERE <CONDITIONAL CLAUSE>;
```

7. Retrieval of Information: (Basic Stuff)

```
SELECT NATIONAL_ID FROM CITIZEN WHERE AGE>21 AND  
COUNTRY='Bangladesh';
```

8. Primary Key and Secondary Key Sample:

```
CREATE TABLE COURSE  
(  
    COURSE_ID VARCHAR2 (8) ,  
    TITLE VARCHAR2 (30) ,  
    PROGRAM VARCHAR2 (5) ,  
    DEPT_NAME VARCHAR2 (20) ,  
    CREDITS NUMBER ,  
    CONSTRAINT PK_COURSE PRIMARY KEY ( COURSE_ID , PROGRAM ) ,  
    CONSTRAINT FK_COURSE_DEPARTMENT FOREIGN KEY( DEPT_NAME )
```

```
REFERENCES DEPARTMENT ( DEPT_NAME ) ON DELETE CASCADE
);
```

9. Distinct:

```
SELECT DISTINCT DEPT_NAME FROM EMPLOYEE ;
```

10. BETWEEN:

The BETWEEN operator selects values within a given range. The values can be numbers, strings or dates. The BETWEEN operator is inclusive: begin and end values will be included. However, you have to provide the lower value first then the upper value.

```
SELECT NAME , DEPT_NAME , SALARY
FROM EMPLOYEE
WHERE SALARY BETWEEN 35000 AND 80000 ;
```

11. IN:

The IN operator allows one to specify multiple values in WHERE clause. It is a shorthand for multiple OR conditions.

```
SELECT NAME , SALARY
FROM EMPLOYEE
WHERE DEPT_NAME in ('DEV ', 'TESTING ');
```

12. Rename:

SQL aliases are used to give a table, or a column in a table, a temporary name. We can also use it for renaming an expression or an entire query.

```
SELECT ID , NAME , ( SALARY *12) AS Y_SALARY
FROM EMPLOYEE;
```

13. LIKE:

1. This will show you all the names that start with capital A:

```
SELECT NAME
FROM EMPLOYEE
WHERE NAME LIKE 'A%';
```

2. For restricting the name size of the resultant name additionally, let's say 4 characters:

```
SELECT NAME
FROM EMPLOYEE
WHERE NAME LIKE 'A___ ';
```

3. More examples:

```
SELECT * FROM employees WHERE name LIKE 'J%'; -- Names starting with 'J'
SELECT * FROM employees WHERE name LIKE '%son'; -- Names ending with 'son'
SELECT * FROM employees WHERE name LIKE '_ohn'; -- Names like 'John', 'Rohn', etc.
SELECT * FROM employees WHERE name LIKE '%o_e%'; -- Names containing 'o' followed by any char, then 'e'
```

14. Concatenation:

Used to combine two or more strings.

Example:

```
SELECT first_name || ' ' || last_name AS full_name
FROM employees;
```

15. More into String:

1. REPLACE: replaces occurrences of a substring with another string.

```
SELECT name, REPLACE(phone, '-', ' ') AS formatted_phone FROM users;
```

name	phone	formatted_phone
Alice	123-456-7890	123 456 7890
Bob	987-654-3210	987 654 3210

2. LTRIM: Removes spaces or specific characters from the left side of a string.

```
SELECT name, LTRIM(employee_code, 'E') AS code_number FROM users;
```

name	employee_code	code_number
John	E12345	12345
Smith	E56789	56789

3. RTRIM: Removes spaces or specific characters from the right side of a string.

```
SELECT name, RTRIM(username, 'X') AS clean_username  
FROM users;
```

name	username	clean_username
Alice	AlexXX	Alex
Bob	BobX	Bob

4. Trim: Removes both leading and trailing spaces or specific characters.

```
SELECT id, name, TRIM(name) AS clean_name FROM users;
```

id	name	clean_name
1	' Alice '	'Alice'
2	'Bob '	'Bob'
3	' Charlie '	'Charlie'
4	'David'	'David'

5. Length: Returns the number of characters in a string (including spaces).

```
SELECT name, LENGTH(name) AS name_length FROM users;
```

name	name_length
Alice	5
Bob	3
John Smith	10

6. LPAD: Pads the left side of a string with a specified character until it reaches a given length.

```
SELECT name, LPAD(id, 6, '0') AS formatted_id  
FROM users;
```

name	id	formatted_id
Alice	45	000045
Bob	7	000007

7. RPAD: Pads the right side of a string with a specified character until it reaches a given length.

```
SELECT name, RPAD(name, 10, '.') AS padded_name  
FROM users;
```

name	padded_name
Alice	Alice.....
Bob	Bob.....

16. GROUP BY & HAVING:

Feature	GROUP BY	HAVING
Purpose	Groups rows based on column values	Filters groups after aggregation
Works On	Columns directly	Aggregated values (<code>SUM()</code> , <code>COUNT()</code> , etc.)
Used With	<code>SELECT</code> , <code>HAVING</code> , <code>ORDER BY</code>	Always used with <code>GROUP BY</code>
Filters	Individual rows before grouping	Grouped results after aggregation

emp_id	department	amount
1	IT	500
2	IT	700
3	HR	400
4	HR	300
5	Sales	1000

```
SELECT department, SUM(amount) AS total_sales FROM sales
GROUP BY department;
```

department	total_sales
IT	1200
HR	700
Sales	1000

```
SELECT department, SUM(amount) AS total_sales FROM sales
GROUP BY department HAVING SUM(amount) > 1000;
```

department	total_sales
IT	1200

Clause	Works Before or After Grouping?	Filters On
<code>WHERE</code>	Before Grouping	Individual rows
<code>HAVING</code>	After Grouping	Aggregated values

```
SELECT department, SUM(amount) AS total_sales FROM sales WHERE amount > 400 -- Removes HR (300) GROUP BY department;
```

department	total_sales
IT	1200
Sales	1000

```
SELECT department, SUM(amount) AS total_sales FROM sales
GROUP BY department HAVING SUM(amount) > 400; -- Filters after grouping
```

department	total_sales
IT	1200
HR	700
Sales	1000

17: Aggregate Functions:

Function	Description
<code>SUM()</code>	Adds all values in a column
<code>AVG()</code>	Calculates the average of values
<code>COUNT()</code>	Counts the number of rows

Function	Description
MAX()	Returns the highest value
MIN()	Returns the lowest value
STDDEV()	Returns the standard deviation
VARIANCE()	Returns the variance
MEDIAN()	Returns the median value
LISTAGG()	Concatenates values into a single string

```
SELECT department,
       COUNT(*) AS num_sales,
       SUM(amount) AS total_sales,
       AVG(amount) AS avg_sales,
       MAX(amount) AS max_sale,
       MIN(amount) AS min_sale,
       MEDIAN(amount) AS median_sale,
       VARIANCE(amount) AS sales_variance,
       STDDEV(amount) AS sales_stddev
FROM sales
GROUP BY department;
```

department	num_sales	total_sales	avg_sales	max_sale	min_sale	median_sale	sales_variance	sales_stddev
IT	2	1200	600.00	700	500	600	20000.00	141.42
HR	2	700	350.00	400	300	350	5000.00	70.71
Sales	2	2500	1250.00	1500	1000	1250	125000.00	353.55

18. Handling Null Values:

Example 1:

```
SELECT emp_id, emp_name, NVL(salary, 5000) AS updated_salary
FROM employees;
```

emp_id	emp_name	salary	updated_salary
101	Alice	8000	8000
102	Bob	NULL	5000
103	Carol	6000	6000

Example 2:

```
SELECT emp_id, emp_name, NVL2(salary, 'Active', 'Pending') AS status
FROM employees;
```

emp_id	emp_name	salary	status
101	Alice	8000	Active
102	Bob	NULL	Pending
103	Carol	6000	Active

19. JOIN

1. Inner Join

Returns only the rows where there is a match in both tables.

```
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

2. Left Join (All left + Matching Right)

```
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

3. Right Join (All Right + Matching Left)

```
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM employees e
RIGHT OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

4. Full Outer Join (All Left + All Right)

```
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

5. Cross Join (Cartesian Product)

```
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

6. Self Join

```
SELECT e1.emp_name AS employee, e2.emp_name AS manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.emp_id;
```

PL/SQL

Block Structure

```
[ DECLARE ]
declaration_statements
BEGIN
execution_statements
[ EXCEPTION ]
exception_handling_statements
END
```

Hello_World

```
SET SERVEROUTPUT ON SIZE 1000000
BEGIN
DBMS_OUTPUT . PUT_LINE ( 'Hello , World ! ');
END ;
/
```

Conditionals – Sample

```
DECLARE
num NUMBER := 7;
BEGIN
IF MOD(num, 2) = 0 THEN
DBMS_OUTPUT.PUT_LINE('The number is even.');
```

```
ELSE
DBMS_OUTPUT.PUT_LINE('The number is odd.');
```

```
END IF;
END;
/
```

Functions – Sample

```
CREATE FUNCTION function_name (parameter_name datatype, ...)
RETURN return_datatype
IS
```

```

-- Variable declarations
variable_name datatype;
BEGIN
-- Function logic (SQL queries, calculations, conditions)
RETURN return_value; -- The function must return a value
EXCEPTION
-- Exception handling (optional)
WHEN others THEN
RETURN error_value;
END function_name;
/

-- Example:
CREATE FUNCTION get_increment(emp_id NUMBER)
RETURN NUMBER
IS
emp_salary NUMBER;
increment NUMBER;
BEGIN
-- Get employee salary
SELECT salary INTO emp_salary FROM employees WHERE employee_id = emp_id;
-- Apply increment logic
IF emp_salary > 5000 THEN
increment := emp_salary * 0.10; -- 10% increment
ELSE
increment := emp_salary * 0.20; -- 20% increment
END IF;
RETURN increment;
EXCEPTION
WHEN NO_DATA_FOUND THEN
RETURN 0; -- Return 0 if employee not found
END get_increment;
/

Calling the Function:
SELECT get_increment(101) FROM dual;

```

Procedure – Sample

```

CREATE PROCEDURE procedure_name (param1 IN datatype, param2 OUT datatype, ...)
IS
-- Variable declarations
variable_name datatype;
BEGIN
-- Executable statements (SQL queries, calculations, conditions)
-- Use param1 and set values for OUT parameters
EXCEPTION
-- Exception handling (optional)
WHEN others THEN
-- Handle errors
END procedure_name;
/

Example: A Procedure to Find the Minimum of Two Numbers
CREATE PROCEDURE findMin (x IN NUMBER, y IN NUMBER) IS
z NUMBER; -- Declare the variable z
BEGIN
IF x < y THEN
z := x;
ELSE
z := y;
END IF;
dbms_output.put_line('Minimum is: ' || z);
END;
/

Executing the Procedure:
DECLARE
a NUMBER;
b NUMBER;
BEGIN
a := 23;
b := 45;
findMin(a, b);
END;
/

```

Loops – Sample

```

WHILE Loop:
DECLARE
-- Variable declarations
BEGIN
WHILE condition LOOP
-- Statements to execute
END LOOP;
END;
/
FOR Loop:
BEGIN
FOR loop_variable IN start_value..end_value LOOP
-- Statements to execute
END LOOP;
END;
/

```

```

DECLARE
    v_counter NUMBER := 1; -- Declare a counter variable
BEGIN
    WHILE v_counter <= 5 LOOP -- Loop while counter is less than or equal to 5
        DBMS_OUTPUT.PUT_LINE('Number: ' || v_counter); -- Print the current number
        v_counter := v_counter + 1; -- Increment the counter
    END LOOP;
END;
/

```

```

BEGIN
    FOR v_counter IN 1..5 LOOP -- Loop from 1 to 5
        DBMS_OUTPUT.PUT_LINE('Number: ' || v_counter); -- Print the current number
    END LOOP;
END;
/

```

Random Practice Query:

```

SELECT p.name, p.type, p.hp
FROM pokemon p
WHERE p.hp = (
    SELECT MAX(p2.hp)
    FROM pokemon p2
    WHERE p2.type = p.type
);

SELECT name, type, (hp + attack + defense + speed) AS battle_potential
FROM pokemon
ORDER BY battle_potential DESC;

SELECT attack_name
FROM attacks
WHERE attack_name LIKE '% %'
GROUP BY attack_name
HAVING COUNT(DISTINCT attack_name) >= 2;

SELECT DISTINCT t.first_name, t.last_name
FROM trainer t
JOIN owner o ON t.trainer_id = o.trainer_id
JOIN pokemon_attacks pa ON o.pokemon_id = pa.pokemon_id
JOIN attacks a ON pa.attack_id = a.attack_id
WHERE a.attack_type = 'Fire'
AND a.attack_power > 50;

```